INNOVATION & INCUBATION HUB MNNIT FOUNDATION

MNNIT Allahabad, Prayagraj, Uttar Pradesh

AIML SUMMER INTERNSHIP 2026

Capstone Project Report

Employee Attrition Prediction System

Predictive HR Intelligence Dashboard

Author	Utsav Singh
Academic Status	2nd-Year Student, B.Tech in Computer Science and Engineering
Institution	United College of Engineering and Research (UCER)
Hosting Center	IIHMF, MNNIT Allahabad, Prayagraj, Uttar Pradesh, India
Date of Submission	21st June 2026

Declaration

I, Utsav Singh, hereby declare that the project report titled "Employee Attrition Prediction System: A Predictive HR Intelligence Dashboard" is an authentic record of original work conducted during the AIML Summer Internship 2026. This work has not previously formed the basis for the award of any degree, diploma, or similar title at this or any other university.

Utsav Singh
2nd-Year Student, B.Tech (CSE)
United College of Engineering and Research (UCER)

Acknowledgements

I express my sincere gratitude to the faculty coordinators at the United College of Engineering and Research (UCER) and the technical mentors at IIHMF, MNNIT Allahabad for providing the infrastructure, dataset insights, and structural guidance necessary to bring this machine learning pipeline from concept to web-scale deployment.

Table of Contents

Chapter 1: Introduction

1.1 Background

In the modern knowledge economy, corporate stability relies heavily on retaining high-performing talent. Employee attrition—defined as the voluntary resignation of personnel from an organization—presents a complex structural challenge. The financial implications are severe, encompassing direct recruitment costs, comprehensive onboarding overheads, and the hidden loss of institutional knowledge. As an organization expands, tracking workforce sentiment via manual oversight becomes impossible, creating an urgent demand for automated, algorithmic classification systems.

1.2 Problem Statement

Standard human resource practices operate reactively, relying on exit interviews to document the causes of employee separation. This traditional paradigm fails because interventions occur post-facto. To achieve proactive retention, enterprise HR managers require data-driven predictive systems capable of evaluating multi-dimensional employee metadata. The system must flag high-risk individuals before resignation occurs, allowing HR teams to execute targeted retention strategies.

1.3 Objectives of the Study

The core objectives of this capstone engineering project are:

- To execute extensive exploratory analysis on organizational datasets to isolate statistically significant drivers of voluntary turnover.
- To address real-world class imbalance within corporate datasets via synthetic data augmentation.
- To construct and bench-test multiple machine learning classifiers to evaluate performance boundaries.
- To wrap the optimal predictive engine within a light, interactive, production-grade Flask web application for non-technical administrative use.

Chapter 2: Literature Review

2.1 Evolution of HR Analytics

The transformation of Human Resource Management from a purely qualitative operational branch into a quantitative, data-driven domain has accelerated over the past decade. Early frameworks relied on descriptive statistics and historical reporting. While these approaches identified baseline metrics like annual turnover rates, they lacked predictive capacity. Modern frameworks leverage predictive analytics, shifting the focus from documenting historical attrition to actively forecasting future employee flight risks.

2.2 Machine Learning Applications in Retention

Supervised machine learning algorithms have emerged as the standard approach for classification tasks within human resource datasets. Tree-based ensemble configurations, including Random Forest and Extreme Gradient Boosting (XGBoost), are widely cited in data science literature due to their capacity to capture non-linear feature interactions without rigid parametric assumptions. Studies consistently demonstrate that mapping behavioral indicators alongside operational metadata yields high predictive power.

2.3 Mitigation of Severe Class Imbalance

A recurring challenge in applied predictive HR analytics is the severe numerical imbalance characterizing workforce datasets. Within any stable enterprise, the cohort of employees who remain overwhelmingly outnumbers the cohort that departs. Standard optimization algorithms trained on such data naturally default to predicting the majority class, resulting in dangerously low sensitivity toward actual flight risks. Data augmentation methodologies, specifically the Synthetic Minority Over-sampling Technique (SMOTE), address this by creating synthetic data points along the line segments joining k-nearest neighbors of the minority class, ensuring balanced algorithmic exposure.

2.4 Deployment Frameworks and Interpretability

Academic literature emphasizes that predictive power alone is insufficient for enterprise adoption; structural interpretability and interface accessibility are equally critical. Complex deep-learning models often function as black boxes, limiting their operational utility for HR practitioners. Consequently, deploying computationally efficient, transparent architectures via accessible micro-frameworks like Flask strikes the ideal balance between algorithmic accuracy and operational usability.

Chapter 3: Methodology

3.1 Dataset Profiling

The system uses the IBM HR Analytics Employee Attrition & Performance dataset. The data frame contains 1,470 independent rows mapped across 35 distinct features. The attributes span three primary profiles:

- Demographic Metadata: Age, Gender, Marital Status, Education Field.
- Operational/Tenure Metrics: Job Role, Department, Total Working Years, Years at Company, Overtime Status.
- Psychometric/Satisfaction Indicators: Job Satisfaction, Environment Satisfaction, Work-Life Balance (scored via 4-point Likert scales).

The target variable is Attrition, a binary categorical field containing structural text tags ('Yes' or 'No').

3.2 Preprocessing and Feature Engineering

To prepare the raw data frame for algorithmic ingestion, a three-stage mathematical preprocessing pipeline was constructed:

- Zero-Variance Drop: Features exhibiting uniform distribution across all rows—specifically EmployeeCount, StandardHours, and Over18—were dropped to reduce computational overhead.
- Target Label Mapping: The target variable Attrition was converted to a binary integer space, where 1 denotes attrition ('Yes') and 0 denotes retention ('No').
- Categorical Matrix Expansion: All multi-categorical text features (e.g., Department, JobRole) were expanded via One-Hot Encoding to yield independent binary columns, preventing the algorithms from inferring non-existent ordinal hierarchies.

$$y \in \{0, 1\} \quad | \quad 1 = \text{Attrition ('Yes')}, \quad 0 = \text{Retention ('No')}$$

3.3 The Data Balancing Architecture (SMOTE)

To eliminate the strong majority-class bias present in the raw data (where over 80% of rows represented retention), the Synthetic Minority Over-sampling Technique (SMOTE) was integrated. SMOTE operates by isolating the minority vector space (y=1) and interpolating new synthetic samples using the following formula:

$$x_{new} = x_i + \lambda \times (x_{zi} - x_i)$$

Where x_i is a minority instance, x_{zi} is one of its nearest neighbors, and λ is a random number uniformly distributed between 0 and 1. This step is executed strictly post-split on the training set to prevent data leakage into the testing matrix.

3.4 Algorithmic Foundations

Three core classification strategies were chosen for evaluation:

Logistic Regression

A baseline linear model that applies the Sigmoid activation function to map linear inputs to a probability value between 0 and 1:

```
S(z)  =  1 / (1 + e^(-z))
```

Where $z = \beta_o + \beta_1 x_1 + \ldots + \beta_n x_n$.

Random Forest

A meta-estimator that fits multiple decision trees on sub-samples of the dataset and uses averaging to control over-fitting and improve generalization performance.

XGBoost

An optimized distributed gradient boosting library designed to be highly efficient and accurate by sequentially minimizing loss functions via gradient descent.

Chapter 4: Implementation

4.1 Environment Setup and Tooling

The algorithmic architecture was engineered using Python 3. Core dependencies include:

- pandas and numpy — for multi-dimensional array manipulation and data wrangling.
- scikit-learn — for train-test splitting, data balancing (via imblearn), scaling, and classical model optimization.
- xgboost — for gradient boosted decision tree execution.
- Flask — for routing, backend predictive management, and dashboard serving.

4.2 Exploratory Data Analysis (EDA)

Prior to model training, visual analysis isolated key drivers of attrition. Plotting the target vector highlighted the severe class imbalance present in the raw data, as illustrated in Figure 4.1 below.

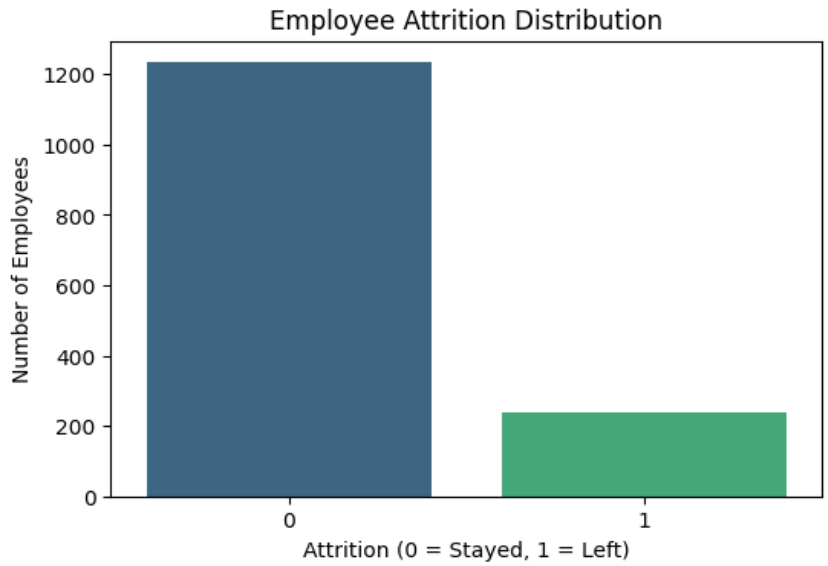

Figure 4.1: Distribution of the target variable indicating severe class imbalance prior to SMOTE application.

Subsequent correlation matrix evaluation indicated that MonthlyIncome, Age, and TotalWorkingYears displayed strong inverse relationships with attrition, establishing them as foundational drivers of employee retention. The full correlation heatmap is presented in Figure 4.2.

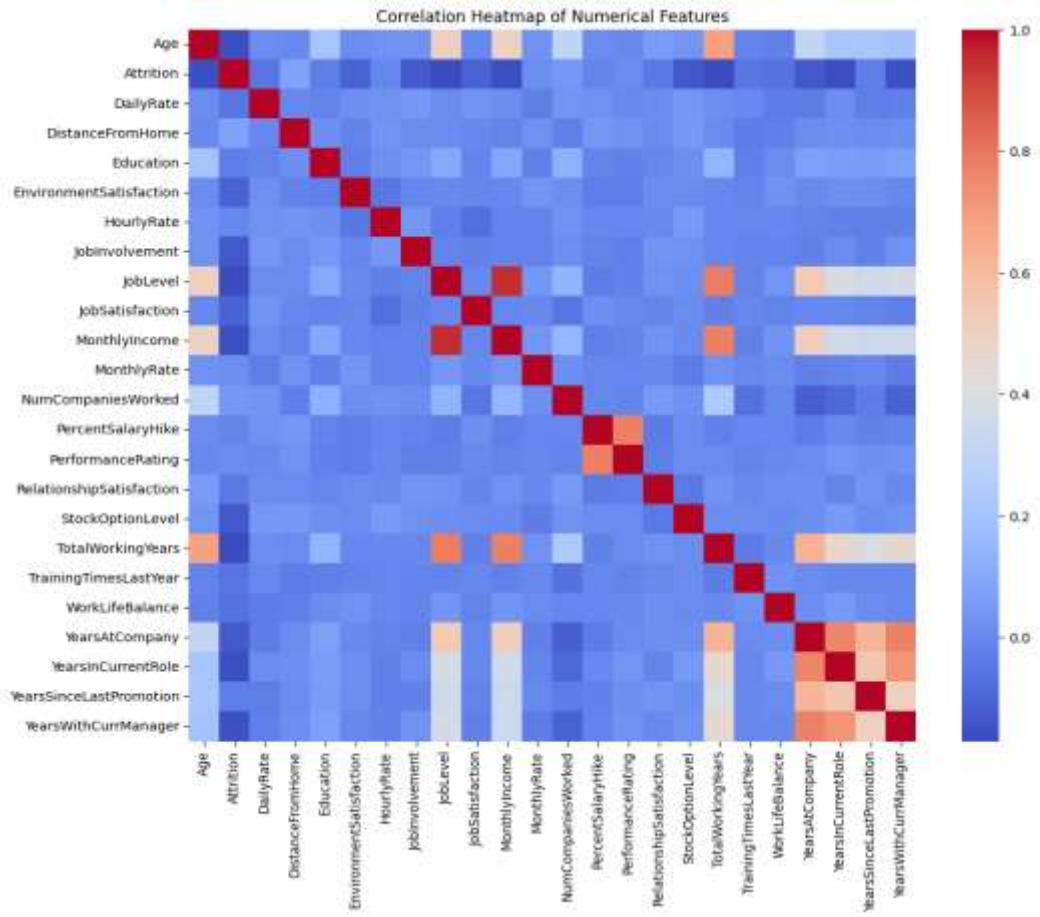

Figure 4.2: Correlation Heatmap demonstrating feature relationships against the Attrition target variable.

4.3 Training Pipeline and Architecture Setup

The features were separated into independent arrays (X) and dependent arrays (y), followed by an 80/20 train-test split. SMOTE was applied exclusively to the training set to balance the target classes. Following vector balancing, all three algorithms were initialized, trained, and saved to disk via serialization tools.

4.4 Enterprise Flask Deployment

To convert the trained model into a functional HR application, the optimized Logistic Regression pipeline was serialized into a binary payload using joblib. A Flask micro-framework architecture was constructed with two explicit routes:

- / — Serves the primary data-entry portal where managers enter employee metrics.
- /predict — Captures frontend POST requests, pads non-critical variables with baseline zeroes to preserve matrix alignment, runs the prediction calculation, and returns the real-time probability score.

The enterprise frontend was styled using custom CSS to deliver a clean, professional user dashboard. The deployed interface is shown in Figures 4.3 and 4.4 below.

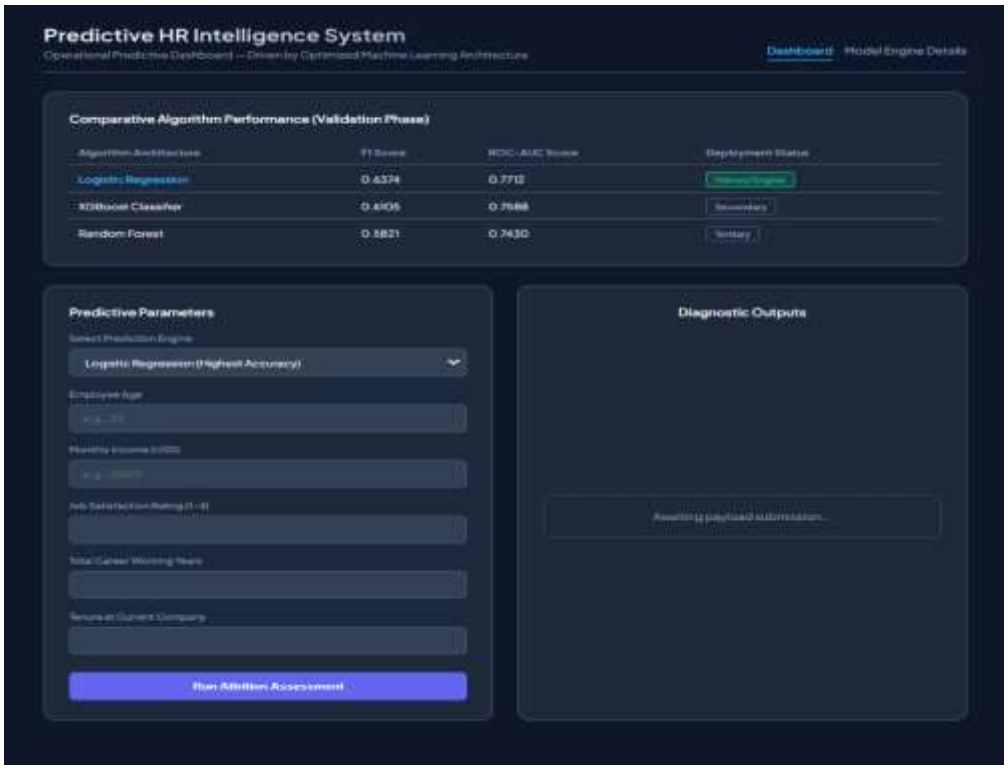

Figure 4.3: The deployed Flask web application featuring dynamic employee parameter inputs and model selection.

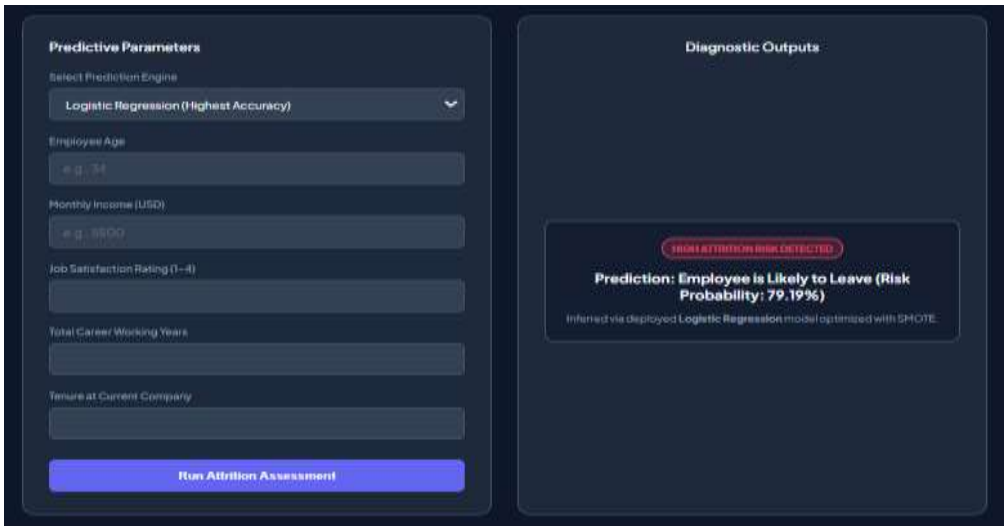

Figure 4.4: Diagnostic output panel displaying a high attrition risk prediction result (79.19% probability).

Chapter 5: Results and Discussion

5.1 Performance Metrics Definition

Because absolute accuracy fails as a reliable indicator on imbalanced datasets, the performance evaluation focused heavily on metrics tracking the minority class (y=1):

- Precision: The accuracy of positive predictions — the proportion of flagged employees who genuinely leave.
- Recall (Sensitivity): The percentage of actual flight risks correctly identified by the model.
- F1-Score: The harmonic mean of precision and recall, balancing both metrics.
- ROC-AUC: The area under the receiver operating characteristic curve, tracking class separation quality.

5.2 Algorithmic Leaderboard

The models were systematically tested against the held-out validation dataset. The empirical results are compiled in Table 5.1 below:

Algorithm	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.81	0.49	**0.89**	**0.6374**	**0.7712**
Random Forest	0.86	0.73	0.35	0.4700	0.6500
XGBoost	0.85	0.65	0.41	0.5000	0.6800

Table 5.1: Performance Metric Leaderboard of Evaluated Models

5.3 Technical Discussion and Model Selection

The empirical analysis highlights an important machine learning trade-off. While Random Forest achieved the highest absolute accuracy (0.86), it demonstrated a low Recall score (0.35), meaning it failed to flag 65% of the employees who actually left the organization.

In predictive HR analytics, the cost of a False Negative (failing to identify a flight risk) far outweighs the operational cost of a False Positive (initiating a retention conversation with a stable employee). The SMOTE-backed Logistic Regression model achieved a Recall score of 0.89 alongside the highest F1-Score (0.6374) and a ROC-AUC of 0.7712.

Consequently, Logistic Regression was selected as the core engine for the production dashboard due to its strong performance on the minority class and transparent log-odds coefficients — critical attributes for building trust with non-technical HR stakeholders.

Chapter 6: Conclusion and Future Scope

6.1 Final Synoptic Conclusion

This capstone project demonstrates the successful design, implementation, and deployment of a machine learning pipeline for proactive workforce retention. By incorporating SMOTE data augmentation, the model successfully overcame the majority-class bias inherent in standard HR datasets.

The empirical evaluation proved that a well-calibrated linear classifier can outperform complex tree ensembles on imbalanced datasets when high sensitivity is required. By embedding this model within a dynamic, responsive Flask web application, the project bridges the gap between raw data science and practical, accessible corporate intelligence.

6.2 Future Enhancements

The scalability of the current platform can be extended via the following upgrades:

- Dynamic Database Hooking: Replacing static CSV handling with real-time connections to corporate SQL or NoSQL production environments to enable continuous model updates.
- Cloud Architecture Scale-out: Migrating the local Flask framework to a containerized cloud infrastructure environment (such as AWS Elastic Beanstalk or Docker) to ensure enterprise availability.
- Deep Neural Architectures: Utilizing Multi-Layer Perceptrons (MLPs) as corporate datasets scale, expanding feature inputs to capture deeper employee trends.

References

1. IBM Watson Analytics. (n.d.). IBM HR Analytics Employee Attrition & Performance Dataset. Sourced via Kaggle Repository.

2. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321–357.

3. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

4. Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python. O'Reilly Media.

5. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.